

NIVASA

Complete Project Interview Questionnaire with Model Answers

Node.js • Express • MongoDB • EJS • Gemini AI

PLACEMENT PREPARATION HANDBOOK

Prepared from the actual project repository
August 2026

NIVASA

Complete Project Interview Questionnaire with Model Answers

How to use this guide

60-second project pitch

"Nivasa is an AI-powered rental and travel-planning platform built with Node.js, Express, MongoDB, Mongoose, EJS, and Bootstrap. It supports two roles: customers discover and request stays, while owners manage listings, bookings, blocked dates, and analytics. I implemented secure session authentication with Passport, Cloudinary image uploads, server-side validation with Joi, overlap-safe availability checks, verified post-stay reviews, and Gemini-powered natural-language search, review summaries, listing recommendations, description generation, and trip planning. The application follows an MVC-oriented structure and uses indexed MongoDB queries for common booking operations."

Architecture snapshot

Layer	Nivasa implementation
Presentation	EJS/EJS-Mate templates, Bootstrap, CSS and browser JavaScript
Routing	Express routers for listings, users, reviews, bookings and AI
Business logic	Controllers plus reusable authentication, authorization and validation middleware
Persistence	Mongoose models: User, Listing, Booking, Review and BlockedDate
External services	MongoDB Atlas, Cloudinary, Google Gemini

Feature flow to describe on a whiteboard

Booking: authenticated customer → Joi validation → date normalization → booking/blocked-date conflict queries → price calculation → pending booking → owner decision → confirmed/completed/cancelled → verified review after completion.

Resume bullets

- Built a role-based rental marketplace with secure booking, availability management and owner analytics using Node.js, Express and MongoDB.
- Designed conflict detection for pending/confirmed reservations and owner-blocked date ranges using indexed, half-open interval queries.
- Integrated Gemini AI for safe natural-language search, review synthesis, travel planning and content generation with output validation and bounded inputs.

1. Project Fundamentals & Architecture

1. What problem does Nivasa solve?

It gives customers one place to discover, compare and request rental stays, while owners manage inventory and booking decisions. AI reduces search friction and assists with descriptions, review insights and trip planning.

2. Why did you choose this technology stack?

JavaScript across client and server reduces context switching. Express is lightweight for REST-style routing, MongoDB fits evolving marketplace documents, Mongoose supplies schemas and relations, EJS enables simple server rendering, and Cloudinary/Gemini provide managed media and AI capabilities.

3. Is this MERN?

Not strictly. The backend uses MongoDB, Express and Node, but the view layer is EJS rather than React. A precise description is a full-stack Node/Express/MongoDB server-rendered application.

4. Explain the MVC structure.

Routes map URLs and middleware; controllers contain request handling and business rules; Mongoose models define persistence; EJS views render responses. This separation improves maintainability and testability.

5. Describe a request lifecycle.

Express parses the request, session and Passport restore the user, route middleware authenticates/authorizes and validates, a controller performs database or service work, then returns HTML/JSON or forwards an error to centralized handling.

6. Why use server-side rendering?

EJS keeps the deployment and data flow simple, gives fast initial HTML and suits a CRUD-heavy product. The trade-off is less rich client state than a SPA; a future React frontend could consume a versioned API.

7. What are the main entities and relationships?

A User is a customer or owner. A Listing belongs to an owner and references reviews. A Booking references a listing, customer and owner. A Review references its author and the completed booking. A BlockedDate references a listing and owner.

8. Why separate controllers and routes?

Routes remain readable declarations of URL, method and middleware order. Controllers can be tested and reused without coupling business logic to routing syntax.

9. What is wrapAsync for?

Express 4 does not automatically forward every rejected async promise. The wrapper catches rejection and calls `next(error)`, allowing one error middleware to respond consistently.

10. What would you redesign first for scale?

Split HTML and API concerns, add a service layer, transactions/atomic availability protection, Redis caching and rate limiting, structured logging, background jobs for AI/media work, automated tests, and production observability.

2. Node.js, Express & API Design

11. Why is Node.js suitable here?

Most work is I/O-bound—database queries, sessions, image upload and AI calls. Node's event loop handles many concurrent waits efficiently. CPU-heavy work should move to workers or a queue.

12. What does middleware mean in Express?

A function receiving `req`, `res`, `next` that can modify context, end the response or pass control. Nivasa uses middleware for parsing, static files, sessions, Passport, flash messages, validation and authorization.

13. Why does middleware order matter?

Later middleware depends on earlier state. Session must run before Passport session restoration; authentication must precede role/ownership checks; upload parsing must precede validation when using multipart forms; the 404 and error handlers must be last.

14. Why use method-override?

HTML forms natively support GET and POST only. A query parameter such as `?_method=PUT` lets forms express edit/delete semantics while Express routes remain REST-like.

15. Which HTTP status codes are important here?

400 invalid input, 401 unauthenticated, 403 unauthorized, 404 missing resource, 409 date/review conflict, 500 unexpected server failure and 502 invalid/unavailable upstream AI response.

16. Why return JSON errors for AI endpoints?

They are called by browser `fetch()`. Returning an HTML error page breaks the client's JSON parsing, so the AI router has local JSON error middleware.

17. What is centralized error handling?

Controllers throw or forward errors with status and message. A final Express handler logs diagnostics and generates one consistent response. It reduces duplicated try/catch response code.

18. How does the 404 handler work?

After all valid routers, a catch-all creates a 404 error and passes it onward. Because it is registered late, only unmatched requests reach it.

19. Why use `Promise.all` in the booking code?

Independent queries—such as booking and blocked-date conflicts, or dashboard datasets—run concurrently, reducing total latency to roughly the slowest query rather than their sum.

20. How would you version an API?

Expose routes under `/api/v1`, define stable request/response contracts, preserve v1 during migration, add deprecation notices, and introduce breaking changes only in v2.

3. MongoDB & Mongoose

21. Why MongoDB?

The domain maps naturally to documents and benefits from flexible iteration. Mongoose adds validation, references, middleware and indexes. The trade-off is that cross-document invariants need extra care.

22. Why reference reviews instead of embedding them?

Reviews have their own author, booking constraint and deletion lifecycle. References avoid unbounded listing documents and allow independent querying, though displaying a listing needs population.

23. What does `populate()` do?

It replaces stored ObjectId references with documents from the referenced collection. It is convenient, but excessive population adds queries/data; projection or aggregation may be better at scale.

24. Explain the compound booking index.

`{ listing:1, checkIn:1, checkOut:1, status:1 }` targets availability queries scoped to one listing and date/status filters. Exact field order should be verified with real queries and `explain()`.

25. Why index status and foreign keys?

Dashboards and conflict checks repeatedly filter by owner, customer, listing and status. Indexes reduce collection scans but cost storage and make writes slightly slower.

26. How is one review per completed booking enforced?

Application logic first checks for an existing review, while a unique sparse index on booking protects the invariant against duplicates at the database level.

27. What is the listing post-delete hook?

After `findOneAndDelete`, it deletes review documents referenced by that listing. It prevents orphaned reviews, but bookings, blocked dates and Cloudinary assets should also be handled in a stronger cleanup design.

28. Difference between Mongoose and Joi validation?

Joi validates the incoming API/form shape early and gives friendly errors. Mongoose protects persisted document rules. Both are valuable because data may enter through paths other than the current HTTP route.

29. Why use `lean()` for location suggestions?

The result is read-only and needs no document methods or change tracking. `lean()` returns plain objects with less memory and conversion overhead.

30. When would you use a transaction?

When several writes must succeed together—for example creating a review and attaching it to a listing, or deleting a listing plus dependent records. Transactions require a replica set and add overhead but prevent partial state.

31. What is a race condition in availability?

Two requests can both pass `exists()` before either inserts. The current pre-check improves UX but is not an atomic guarantee. A production fix could use transactional locking, per-listing mutexes, or an inventory/night model with unique constraints.

32. Why use `exists()`?

Only presence matters, not the full document. It transfers less data and communicates intent clearly.

4. Authentication, Authorization & Security

33. Authentication vs authorization?

Authentication establishes identity; Passport session login handles it. Authorization decides what that identity may do; role and ownership middleware enforce it.

34. How are passwords protected?

Passport-Local-Mongoose supplies salted password hashing and authentication helpers. Plaintext passwords are not stored in the User schema.

35. How do persistent sessions work?

The browser holds an opaque session cookie. Session state is stored in MongoDB through connect-mongo. Passport serializes the user identity into the session and deserializes it on later requests.

36. What do httpOnly, secure and sameSite do?

httpOnly blocks JavaScript cookie access, secure restricts transmission to HTTPS, and sameSite limits cross-site cookie sending. Nivasa sets httpOnly; production should also configure secure and sameSite behind HTTPS.

37. How are ownership checks implemented?

The listing is loaded and its owner ObjectId is compared with req.user._id. Booking mutations additionally query by both booking id and authenticated owner/customer, which prevents horizontal privilege escalation.

38. What is RBAC here?

Role-based access control separates customer and owner capabilities. Owners create/manage listings and availability; customers request stays and submit verified reviews.

39. What security issue exists in the basic country search?

Passing raw user text to \$regex can allow regex metacharacters and expensive patterns. The AI search and suggestions escape regex input; basic search should reuse the same escape function and enforce a length limit.

40. How do you reduce NoSQL injection?

Do not accept arbitrary query objects. Construct allow-listed filters, coerce numbers, validate enums, escape regex input, reject unexpected object shapes and optionally use sanitization middleware.

41. What other protections are missing?

CSRF tokens, Helmet security headers, rate limits—especially for login and AI—secure production cookies, stricter upload validation, request-size limits, secret rotation and structured audit logs.

42. Why should secrets be environment variables?

Database, session, Cloudinary and Gemini credentials must not be committed. Environment injection supports different deployments and rotation. Startup should validate that required variables exist without logging secret values or full connection strings.

5. Booking & Availability Logic

43. Explain the overlap formula.

Two half-open intervals $[start, end)$ overlap when $existingStart < requestedEnd$ and $existingEnd > requestedStart$. Back-to-back stays are allowed because checkout is not treated as an occupied night.

44. Why normalize dates to start of day?

Bookings are night/date based. Removing time components prevents accidental conflicts caused by hours and minutes. For global production use, define the property timezone explicitly rather than relying on server-local time.

45. Which booking statuses block inventory?

Pending and confirmed bookings currently block it. Cancelled and completed do not. Blocking pending requests is conservative but can reduce availability if requests remain unresolved; an expiry policy would help.

46. How is total price calculated?

The server computes nights from normalized dates and multiplies by the listing's stored nightly price. The server—not the browser—is authoritative, preventing client tampering.

47. Why check availability again on confirmation?

State may change after the initial request. Rechecking protects against a blocked period or another booking introduced in the meantime, while excluding the current booking id.

48. How does the owner dashboard compute metrics?

It loads owner bookings, listings and blocked dates concurrently; filters confirmed/completed stays; reduces total revenue and stay days; and estimates occupancy as stay days divided by listing capacity over 30 days.

49. What is inaccurate about current occupancy?

It sums all historical/future stay days rather than clipping reservations to the next 30-day window. A better query intersects each confirmed stay with that reporting window and sums only overlapping nights.

50. How would you add payments?

Create a payment/order record, calculate amount server-side, use a provider checkout, verify signed webhooks idempotently, transition booking state only after verified payment, and handle refunds/cancellations through auditable state changes.

6. Gemini AI Integration

51. What AI features exist?

Listing-description generation, natural-language search filters, destination validation, travel plans, review summaries and similar-stay discovery. Similar listings use deterministic database logic; the others use Gemini where language reasoning adds value.

52. Explain natural-language search end to end.

The user phrase is bounded and sent as quoted data. Gemini returns allow-listed JSON filters. The server strips optional code fences, parses and sanitizes fields, corrects reversed price bounds, builds a safe Mongo query, then redirects to normal search results.

53. How do you defend against prompt injection?

Treat user text as data, demand a narrow JSON schema, allow-list keys/categories, bound lengths and numeric ranges, escape regex, and never execute model output. Stronger production code should use native structured output/schema validation.

54. What if Gemini returns invalid JSON?

Parsing throws and reaches the AI-specific error middleware. The client receives a stable JSON failure. A robust version would retry once with schema enforcement, log a request id, and optionally fall back to keyword parsing.

55. How are hallucinations limited in trip planning?

The prompt prohibits invented opening hours, bookings and prices; validates the destination first; supplies actual matching stays; and asks for activity types rather than unsupported precise facts.

56. Why not use AI for every recommendation?

Deterministic filters for category/location/country and price distance are faster, cheaper and explainable. AI is most useful for ambiguous language; ordinary ranking should remain testable and predictable.

57. How would you control AI cost and latency?

Rate-limit by user/IP, cache normalized searches and review summaries, generate summaries only when reviews change, use compact prompts and fast models, apply timeouts and retries, and move non-interactive generation to background jobs.

58. How would you evaluate the AI features?

Build labeled test sets for filter extraction and destination correction; measure field accuracy, invalid-output rate, latency and cost. For summaries/plans, add groundedness and safety rubrics plus human review and regression tests.

7. Frontend, Validation, Media & UX

59. Why validate on both client and server?

Client validation improves feedback, but it can be bypassed. Server validation is the security and integrity boundary. Database validation adds the final persistence guard.

60. Why Joi?

It declaratively validates nested form payloads, types, limits, enums and cross-field date rules before controller logic runs.

61. How do uploads work?

Multer parses multipart form data and the Cloudinary storage adapter uploads the file. The controller stores Cloudinary URL and filename in the listing.

62. What should be added to upload security?

Allow-list MIME types/extensions, cap file size and count, validate actual file signatures, use authenticated transformations, remove replaced/deleted assets and reject suspicious filenames/metadata.

63. What is EJS-Mate?

A layout engine for EJS that supports shared boilerplate. Navigation, footer and common assets live centrally instead of being duplicated across pages.

64. How does location autocomplete work?

After at least two characters, the server escapes the query, searches location/country case-insensitively, selects only needed fields, uses lean documents, deduplicates results and returns at most six suggestions.

65. Accessibility improvements?

Associate labels with controls, support keyboard navigation and visible focus, provide image alt text, announce async availability/AI results, maintain contrast, avoid color-only status, and test with a screen reader.

8. Testing, Debugging & Deployment

66. What tests would you write first?

Unit tests for date overlap, date validation, price and AI filter sanitation; integration tests for authorization, booking conflicts and verified reviews; end-to-end tests for signup → booking → owner confirmation → completion → review.

67. Give overlap test cases.

Exact same range, partial overlap at start/end, requested range containing existing, existing containing requested, back-to-back checkout/check-in, blocked dates, cancelled bookings, past date and invalid/reversed dates.

68. How do you test controllers?

Use a test database, create fixtures, call routes through an HTTP test client, authenticate a session, assert status/body/redirect and verify database state. Mock Gemini and Cloudinary at service boundaries.

69. How would you debug a slow listing page?

Trace request duration, inspect Mongo queries with `explain()`, check population and document size, verify indexes, measure external calls, profile template/static assets, then optimize based on evidence.

70. What should a production deployment include?

HTTPS, validated environment configuration, secure cookies and trusted proxy settings, managed MongoDB/Cloudinary, process health checks, logs/metrics/error tracking, backups, CI tests, dependency scanning, graceful shutdown and horizontal scaling.

71. Why is `console.log("Mongo URL:", ...)` risky?

A connection string may contain credentials. Logs are widely accessible and retained. Log only whether configuration is present, never the secret value.

72. Why should the demo-user route be removed?

It embeds a predictable credential and creates an account through a public GET request. Production must remove it; demo data belongs in a controlled seed script.

73. How would you add observability?

Structured request logs with correlation ids, latency/error metrics, database and AI timing, exception tracking, health/readiness endpoints and alerts on error rate, latency and availability.

74. How do you handle graceful shutdown?

On termination, stop accepting traffic, allow active requests to finish within a deadline, close the HTTP server and MongoDB connection, then exit. This prevents dropped requests during deploys.

9. Scenario & System-Design Questions

75. Two users book the last dates simultaneously. What do you do?

Explain that pre-checks race. Introduce an atomic reservation mechanism: a transaction with per-listing/date inventory documents and a unique index, or a lock with strict timeout and idempotency. Return 409 to the loser.

76. Traffic grows to one million listings.

Add pagination, geospatial/search indexes, denormalized listing cards, CDN images, Redis caching, replicas, query projections, asynchronous jobs and monitoring. Move fuzzy search to Atlas Search or a dedicated search engine.

77. Gemini is unavailable.

Keep core listing and booking workflows operational. Apply timeouts/circuit breaker, show a clear retry message, fall back to ordinary filters, serve cached summaries, and queue optional generation.

78. How would you add notifications?

Emit domain events when booking status changes, enqueue email/SMS/push jobs, process with retries and idempotency keys, store delivery status, and never make the user request wait on the provider.

79. How would you add property capacity?

Store maxGuests on Listing, validate guests against it server-side, expose it in search, and consider children/rooms only if the domain needs them. Existing global max 20 is not property-specific.

80. How would you implement cancellation policy?

Version policy terms on the booking, calculate refund from server time and check-in timezone, record immutable refund decisions, call the payment provider idempotently and notify both parties.

81. How would you prevent stale pending bookings?

Add an expiresAt timestamp and TTL/background expiry job. Conflict queries should ignore expired requests; notify the user and release inventory automatically.

82. How would you support multiple app servers?

Sessions are already externalized in MongoDB, so servers can remain mostly stateless. Put them behind a load balancer, centralize cache/queues, avoid in-memory locks and coordinate atomic booking state in shared infrastructure.

10. Code Review: Honest Improvements

Current observation	Improvement to propose
Connection string is logged	Remove secret output and validate configuration safely at startup.
Public /demouser route with fixed password	Delete it; create a separate, environment-gated seed script.
Listing edit page lacks ownership middleware	Add isOwner to GET /:id/edit; never rely only on hiding UI.
Basic country search uses unescaped regex	Escape metacharacters, limit length and reuse a shared query builder.
Cookie lacks production secure/sameSite settings	Set secure, sameSite and appropriate proxy/trust configuration in production.
Booking availability uses check-then-insert	Add atomic inventory/locking/transaction strategy and concurrency tests.
Review create performs two writes	Use a transaction or redesign listing-review relation to avoid partial state.
Deleting listing leaves bookings/blocks/media	Define retention rules and transactional/event-driven cleanup, including Cloudinary.
Occupancy is not restricted to 30-day window	Clip each stay to the reporting interval and aggregate only intersecting nights.
No automated tests in package script	Add unit, integration and E2E suites to CI with a test database and mocked services.

A polished “biggest challenge” answer

“The hardest part was availability because it crosses validation, date mathematics, booking states and owner-blocked dates. I modeled stays as half-open intervals, normalized dates, and used two parallel existence queries. I also recheck on confirmation. During review I identified that check-then-insert is still vulnerable to concurrent requests, so the next production step is an atomic inventory strategy with concurrency tests.”

A polished “what did you learn?” answer

“I learned that feature completeness is different from production correctness. Authentication needs authorization; UI validation needs server validation; an AI response needs schema validation; and an availability check needs concurrency protection. The project improved how I think about invariants and failure paths, not only happy-path CRUD.”

11. HR & Behavioral Round

83. Tell me about yourself.

Template Present → evidence → direction. “I am a developer focused on full-stack JavaScript. In Nivasa I built the end-to-end rental workflow and integrated secure authentication, MongoDB data modeling and Gemini features. I enjoy turning business rules into dependable systems, and I’m looking for a role where I can strengthen backend and product engineering skills.”

84. Why did you build Nivasa?

Explain the user problem and technical learning goal: a familiar marketplace domain with non-trivial roles, availability, verified feedback and practical AI—not AI added only for appearance.

85. What was your individual contribution?

Be exact. Name the parts you designed/coded/tested. If it was a team project, credit teammates and explain interfaces and decisions you personally owned.

86. Describe a failure or bug.

Use STAR: situation, task, action, result. Pick a real example such as date overlap, redirect state, AI JSON parsing or authorization; include how you reproduced it and what test would prevent recurrence.

87. How do you handle disagreement?

Clarify the shared goal, compare options using constraints and evidence, run a small experiment when possible, document the decision, and support the chosen path even if it was not your first preference.

88. Why should we hire you?

Connect evidence to the job: end-to-end ownership, practical backend fundamentals, ability to identify limitations honestly, and speed of learning. Avoid generic claims without examples.

89. What would you do with two more weeks?

Prioritize reliability: automated booking/auth tests, atomic reservation protection, CSRF/rate limiting/security headers, accurate analytics, service-layer refactor, CI and production monitoring.

90. Explain the project to a non-technical person.

“It is a website where travelers find and request places to stay and owners manage those requests. It prevents conflicting dates, allows reviews only after a real completed stay, and uses AI to understand natural search phrases and help plan trips.”

12. Rapid-Fire Revision

Final interview checklist

■ 60-second pitch ■ Architecture drawing ■ Booking overlap formula ■ Three security improvements
■ One genuine challenge ■ One bug story ■ Scale plan ■ Clear personal contribution
■ GitHub/README ready ■ Working demo or screenshots ■ Questions for interviewer

Smart questions to ask the interviewer

1. What does success in the first three months look like for this role?
2. How does the team review designs and code before production?
3. Which reliability or scaling challenges is the team working on now?
4. How are junior engineers mentored and given ownership?